STEMQuest

Project Architecture and Technical Documentation

A simple beginner-friendly guide for an M.Tech AI & ML project expo

Interactive STEM learning	Gamified activities	Demo mode + optional Supabase

Project repository supplied by the owner:

https://github.com/ArasiIrini28/STEMQuest

Prepared from the inspected STEMQuest source folder. This guide separates what is implemented now from features that still need external configuration or live verification.

Table of Contents

Section	Page
1. Start here: the project in two minutes	3
2. Technology guide	4
3. Folder and file guide	6
4. How the files connect	9
5. Architecture and data flow	11
6. Student journey and learning logic	12
7. Setup, commands and deployment	13
8. Testing and validation	15
9. Expo preparation	16
10. Viva questions and answers	17
11. Revision sheet	19

Page numbers are generated in the PDF footer. The listed locations help you find each topic quickly; exact page breaks can vary slightly if the PDF viewer substitutes fonts.

1. Start here: the project in two minutes

One-sentence explanation: STEMQuest is a responsive web application that teaches STEM topics through short lessons, quizzes, games, simulations, XP rewards, progress views, and role-based tools.

What a judge should understand

- The visible interface is built in React and delivered by Next.js. The main application shell is `src/components/stemquest.tsx`.
- It has a fully usable **browser-local demo mode**. Demo progress is saved in the browser's localStorage, so it is not a secure multi-user database.
- A Supabase-backed production path is implemented in the source. It needs the project URL, public key, migrations, Auth configuration and live verification before it should be called production-ready.
- The source includes 16 lessons and 30 activities, including robot maze, fraction games, gravity and circuit simulations, and a restricted Python exercise.
- The database design keeps answer keys in a private schema and validates rewards on the server through PostgreSQL functions when Supabase is connected.

Implemented now vs. requires setup

Area	Current state	What to say accurately
Demo learning	Implemented	Runs without credentials; stores progress only on the current browser/device.
Games and simulations	Implemented	Robot maze, fuel game, memory pairs, gravity lab, circuit lab and limited Python playground.
Accounts and cloud records	Implemented in source; needs setup	Supabase Auth/client code and secure SQL exist. A real Supabase project must be configured and tested.
Teacher, guardian, admin tools	Partly demo; database path implemented	Demo tools are local. Connected roles depend on authenticated data and operator setup.
Email reminders and AI tutor	Optional integration; unverified live	Edge Functions are present but require secrets, service configuration and live testing.
Vercel deployment	Configured, not claimed verified here	`vercel.json` exists. You must connect your own Vercel account and configure variables.

2. Technology guide

A technology is simply a tool used to solve one part of the project. The table below explains the tools that are actually declared in `package.json` or configuration files.

Technology	What it is	Why STEMQuest uses it	Where you see it
Next.js 16	A React framework that builds and serves web apps.	Provides the application structure and generates the static `out/` website.	`src/app/`, `next.config.ts`, npm scripts
React 19	A UI library made of components.	Keeps the dashboard, games and pages as reusable interactive pieces.	Every `src/components/*.tsx` file
TypeScript	JavaScript with type checking.	Catches incorrect data shapes before the app runs.	`src/lib/types.ts`, `tsconfig.json`
CSS + Tailwind import	Rules that control appearance. Tailwind is imported, while most UI rules are custom CSS.	Preserves the purple space-adventure visual design and responsive layout.	`src/app/globals.css`
Framer Motion	Animation library.	Controls motion preferences and UI animation support.	`stemquest.tsx`, `activities.tsx`
Lucide React	Icon library.	Provides accessible interface icons instead of hand-drawn icon code.	Most component files
Supabase JS	Browser client for Supabase.	Calls Auth and database RPC functions when public configuration is present.	`src/lib/supabase.ts`
PostgreSQL / Supabase	Cloud relational database.	Stores profiles, lessons, progress, rewards, classes and private answer keys.	`supabase/migrations/*.sql`
SQL migrations	Versioned database change files.	Create the schema and database functions in a repeatable order.	`001` through `005` migrations
Supabase Edge Functions	Server-side Deno functions hosted by Supabase.	Implement optional reminders, tutor access and account actions without exposing secrets to the browser.	`supabase/functions/`
npm	Node package manager.	Installs locked dependencies and runs standard project commands.	`package.json`, `package-lock.json`
Vitest + PGlite	Unit and embedded PostgreSQL test tools.	Tests learning logic and SQL/RLS behavior without a live cloud database.	`tests/engine.test.ts`, `tests/database.test.ts`
Playwright + Axe	Browser automation and accessibility checks.	Exercises important learner journeys in Chromium and scans key accessibility rules.	`tests/e2e/journey.spec.ts`
ESLint / TypeScript check	Static code-quality tools.	Find style, React and type errors before building.	npm scripts
Git / GitHub	Version control and source hosting.	Lets you track changes, collaborate and connect deployment platforms.	Repository history / remote repository
Vercel config	Hosting configuration.	Describes how a Vercel project should install, build and publish the static output.	`vercel.json`

How the technologies work together

Student browser	Next.js + React UI	TypeScript learning logic	Supabase client (optional)	PostgreSQL + Edge Functions

Demo path: Browser → React components → engine functions → localStorage. **Connected path:** Browser → Supabase client → protected SQL RPCs → PostgreSQL tables. The browser never receives private answer-key tables or a service-role secret.

Important vocabulary

Term	Simple meaning in this project
Component	A reusable React UI part, such as a dashboard or game.
State	Current information kept while the app is running, such as XP or selected lesson.
Type	A TypeScript description of the data a function expects.
Migration	A numbered SQL file that changes the database in a known order.
RLS	Row Level Security: database rules that decide which rows each logged-in user may access.
RPC	A database function called from the app. In STEMQuest, it checks answers and awards XP.
Edge Function	A server endpoint used for privileged or external-service work.

3. Folder and file guide

Read this section with VS Code open. Start at the top-level folder, then open the folders in this order: `src/app` → `src/components` → `src/lib` → `supabase` → `tests`.

Top-level project files

File / folder	Purpose
`src/app/`	Next.js entry area. It provides the HTML shell, page entry and global styling.
`src/components/`	Visible UI blocks: dashboard, activities, accounts, staff tools, games and artwork.
`src/lib/`	Shared non-visual logic: types, content, rewards, demo storage and Supabase client.
`supabase/`	Database configuration, SQL migrations and optional server-side Edge Functions.
`tests/`	Automated unit/database and browser journey tests.
`scripts/`	Small Node utilities: create seed SQL and serve built static files locally.
`docs/`	Feature status, security, Supabase, services, deployment and Windows setup documents.
`package.json`	Project name, exact npm commands and direct dependency versions.
`package-lock.json`	Exact dependency tree used by `npm ci`; do not hand-edit it.
`.env.example`	Safe list of variable names. It contains placeholders, never real credentials.
`next.config.ts`	Requests static export, disables image optimization and configures the TypeScript CLI behavior.
`tsconfig.json`	TypeScript compiler settings and the `@/*` import alias.
`playwright.config.ts`	Browser-test base URL, server behavior, timeouts and report settings.
`vitest.config.ts`	Runs the unit/database tests from `tests/*.test.ts`.
`vercel.json`	Vercel build/output settings and security headers.
`README.md`	Main project instructions, project map and honest scope statement.

The Next.js application files

File	What it does	If it were missing
`src/app/layout.tsx`	Defines root HTML/body, title and metadata; imports global CSS.	The app would lose the common document shell and global stylesheet import.
`src/app/page.tsx`	Very small entry page. It renders `<StemQuest />`.	There would be no main page component to show.
`src/app/globals.css`	Defines colors, layout, responsive styles, form/button styles and game/circuit visuals.	The app would still have logic but look largely unstyled and may become difficult to use.

Component files

File	Real responsibility
`stemquest.tsx`	Main controller. Holds view, learner state, demo/connected decision, navigation, loading/saving progress and routing to all screen components.
`landing.tsx`	Welcome/marketing screen using subject data and decorative artwork.
`dashboard.tsx`	Dashboard cards, world catalogue, lesson cards and recommendations.
`activities.tsx`	Lesson player and activity interfaces; also GravityLab and limited PythonLab.
`games.tsx`	FractionFuel, FractionMemory and CircuitLab. It exposes the circuit calculation helper too.
`insights.tsx`	Missions, achievements, progress analytics, leaderboard and tutor screen.
`account.tsx`	Auth, onboarding and settings forms. It gives a clear message when Supabase is not configured.
`staff.tsx`	Teacher, guardian and admin panels; local demo tools or Supabase RPC calls depending on connection.
`content-studio.tsx`	Admin editor for a one-question custom lesson; saves draft/published lessons.
`art.tsx`	Reusable SVG/visual art for world cards and hero areas.

The shared library files

File	Real responsibility	Main users
`types.ts`	Defines Level, ActivityKind, Activity, Lesson, World, Attempt, Profile and LearningState.	Components, engine, content, demo and custom lessons.
`content.ts`	Defines the six subject worlds, built-in lessons, activities and achievement thresholds. Imports extra lessons.	Dashboard, insights, staff, landing, stemquest, seed generator, tests.
`extra-content.ts`	Adds spaceship fuel and fraction-memory lessons to the content list.	content.ts
`engine.ts`	Default state, date/streak calculation, maze validation, restricted Python parser, answer checking and XP/coin reward logic.	stemquest, activities, dashboard, insights, engine tests.
`demo.ts`	Reads the local demo roster and calculates ranking rows from earned activity.	stemquest and insights leaderboard.
`custom-lessons.ts`	Loads/saves local demo authored lessons or connected Supabase lessons.	stemquest and content-studio.
`supabase.ts`	Creates a public Supabase client only when both public values are valid. Gives a readable config error otherwise.	stemquest, account, insights, staff, custom-lessons.

4. How the files connect

This is the central map of the frontend. Every arrow below is supported by an actual import or direct component use in the inspected source.

layout.tsx + globals.css	page.tsx	stemquest.tsx	screen components	library logic + data

Connection	What passes across it	Why it is needed / what breaks without it
`layout.tsx → globals.css`	The global CSS file is imported once.	All screens share a consistent design. Without it, UI styling disappears.
`page.tsx → stemquest.tsx`	Page renders the main `StemQuest` React component.	This is the entry bridge from Next.js into the actual application.
`stemquest.tsx → dashboard.tsx`	Passes learner state and navigation callbacks to Dashboard, WorldCatalogue and LessonCard.	Dashboard cannot show XP, lessons or open a lesson without these inputs.
`stemquest.tsx → activities.tsx`	Passes current lesson and a `submit` callback to LessonPlayer; renders GravityLab/PythonLab.	Activity answers cannot update reward state or reach server validation.
`activities.tsx → games.tsx`	Imports FractionFuel and FractionMemory. Each receives submit/busy props.	Special game activity kinds would have no interactive screen.
`stemquest.tsx → games.tsx`	Renders CircuitLab under science simulations.	The circuit simulation would not be reachable.
`stemquest.tsx → account.tsx`	Uses Auth, Onboarding and Settings; passes save/reset callbacks.	No sign-in/onboarding/settings screen.
`stemquest.tsx → insights.tsx`	Uses Missions, Achievements, Analytics, Leaderboard and Tutor with learner state.	No progress views, rankings or tutor UI.
`stemquest.tsx → staff.tsx`	Passes demo flag, state and current lesson list.	Teacher/guardian/admin surfaces cannot adapt to demo vs connected mode.
`staff.tsx → content-studio.tsx`	Admin view renders ContentStudio.	No lesson-authoring editor in the admin panel.
`content-studio.tsx → custom-lessons.ts`	Loads and saves authored lesson records.	The editor could not persist a draft or published lesson.

Data and logic connection guide

Connection	What happens
`content.ts → extra-content.ts`	content.ts imports the two extra lesson objects and appends them to the main lesson catalogue.
`components → types.ts`	Type imports make sure every screen agrees on lesson, activity and state shapes.
`stemquest.tsx → engine.ts`	For demo submissions it calls `validateAnswer` and `award`; for display it uses date/streak helpers.
`activities.tsx → engine.ts`	The maze UI uses `runMaze`; the coding UI uses `runPythonSubset`. This lets the preview match the demo reward logic.
`dashboard.tsx / insights.tsx → content.ts`	They read built-in worlds, lessons and achievements to list content and calculate display progress.
`stemquest.tsx / insights.tsx → demo.ts`	They read a browser-local roster and compute rankings from recorded XP attempts.
`account.tsx / staff.tsx / insights.tsx / custom-lessons.ts → supabase.ts`	These files share the one optional public Supabase client. If absent, code shows demo or a clear setup message rather than crashing.
`scripts/seed.mjs → content.ts`	The generator reads typed curriculum data and writes reviewable SQL to `work/seed.sql`.
`tests/engine.test.ts → engine.ts + content.ts`	Checks pure reward, safety and streak functions.
`tests/database.test.ts → migrations`	Creates an embedded PostgreSQL database, applies SQL migrations and tests RLS/RPC behavior.
`tests/e2e/journey.spec.ts → built browser UI`	Drives actual pages to test learning, settings, games, content publishing and accessibility.

Remember: **imports** are direct code-to-code dependencies. The SQL migrations and Edge Functions are connected at deployment time through Supabase configuration, not imported by the React browser code.

5. Architecture and data flow

STEMQuest has two real operating modes. This is important to explain correctly at the expo.

A. Demo mode: works immediately

Student browser	React components	engine.ts validation + award()	localStorage demo state

When there is no signed-in Supabase session, `demo = !userId` in `stemquest.tsx`. The app loads `stemquest-demo-v1` from localStorage, checks answers in the browser, and saves XP/progress back to the same browser. It is intentionally useful for a demo but is not secure shared storage.

B. Connected Supabase mode: production path

Student browser	supabase.ts client	Authenticated RPC / RLS	PostgreSQL public + private schemas

When a valid session and public Supabase configuration exist, the React app calls RPCs such as `learning_snapshot`, `save_profile` and `submit_activity`. The database function—not the browser—checks the private answer key and awards XP once.

Security path in simple words

Layer	What the source does
Public browser code	Contains only public Supabase URL/anon key. It never contains a service-role key.
RLS policies	Allow a student to see their own records; allow a teacher/guardian only authorized learner data; restrict private tables.
private schema	Stores answer keys, invitation records, email logs and AI usage logs. Browser roles do not get direct table access.
Security-definer RPCs	Use fixed search paths and identity checks to validate answers, profiles, class invites and authoring.
Edge Functions	Use server-only environment variables for email/AI/service actions. They need real secrets and provider setup before use.

Database migration map

Migration	Purpose
001_core.sql	Creates tables, private schema, RLS policies, account/profile/reward/class/guardian functions.
002_seed.sql	Original seed curriculum and answer keys.
003_services.sql	Reminder delivery claims, unsubscribe flow and AI-use reservation limit.
004_expanded_content.sql	Adds/updates expanded lesson content, including new games.
005_content_and_classrooms.sql	Adds teacher class performance function and secure custom lesson save/publish RPC.

6. Student journey and learning logic

Example: a learner completes “Robot rescue mission” in demo mode

Step	Actual files involved	What happens
1. Open app	page.tsx → stemquest.tsx	The main component loads saved demo state and reads the URL hash view.
2. Browse content	dashboard.tsx + content.ts	The learner opens a world or lesson card using the built-in lesson catalogue.
3. Start lesson	stemquest.tsx → activities.tsx	LessonPlayer receives the lesson and a submit callback.
4. Build route	activities.tsx → engine.ts	Maze controls collect direction commands; runMaze checks grid boundaries and rocks.
5. Submit	stemquest.tsx → engine.ts	Demo validates the answer and uses award() to add one-time XP, coins, completed state and a day record.
6. Review	insights.tsx + dashboard.tsx	Mission, achievement, analytics and leaderboard views read the updated state.

What “gamification” means here

- **XP:** points awarded for a newly mastered scored activity. Duplicate completion receives 0 extra XP.
- **Coins:** derived from XP in the demo engine; the connected SQL function also writes a coin transaction.
- **Streak:** counts activity dates using the learner’s selected IANA timezone. Only a newly mastered scored activity adds a day.
- **Missions:** UI targets based on newly mastered activities; the source does not claim separate extra XP for missions.
- **Achievements:** threshold-based badges. Connected mode inserts earned achievement records when completion thresholds are met.
- **Leaderboard:** demo ranking sums actual local activity XP; connected mode uses class-aware database leaderboard logic.

Interactive learning modules that genuinely exist

Module	What the learner can do	Limit
Quiz/activity types	Choice, multi-select, fill, matching, ordering, flashcards and reflection.	Flashcards/reflections are ungraded practice.
Robot maze	Add commands, run, reset and earn XP on reaching the goal.	The grid/rules are intentionally small.
Fraction games	Fill six of eight fuel cells; find equivalent fraction pairs.	Fixed educational scenarios, not a game engine.
Gravity lab	Switch Earth/Moon and adjust height to see calculated fall time.	Educational model, not a full physics simulator.
Circuit lab	Open/close switch, change voltage/resistance and observe I=V/R and P=V×I.	Ideal DC constant-resistance model.
Python lab	Use supported range-loop/print(i) syntax.	Not a general Python runtime; no arbitrary execution.

7. Setup, commands and deployment

Safe starting point: use Node.js 22.18+ (Node 24 was used in the project checks). The browser demo needs no API key.

Command	What it does	When to use it
`npm install`	Downloads dependencies using the version ranges in package.json.	Normal first setup; for reproducible work prefer npm ci.
`npm ci`	Installs exactly what package-lock.json specifies.	Best for clean setup, CI and repeatable testing.
`npm run dev`	Starts the Next.js development server on port 3000.	While editing. Open http://localhost:3000.
`npm run build`	Creates optimized static output in out/ with Webpack.	Before deployment or previewing the production build.
`npm run start`	Runs scripts/serve.mjs to serve the out/ directory.	To preview the built static website.
`npm run typecheck`	Runs next typegen then TypeScript without emitting JS.	Before committing; catches type errors.
`npm run lint`	Runs ESLint over src and tests.	Before committing; catches code-quality issues.
`npm test`	Runs Vitest unit/database tests.	To check rewards, safety and database rules.
`npm run test:e2e`	Runs Playwright browser journeys; can start dev server automatically.	To test visible user flows.
`npm run seed:generate`	Reads content.ts and writes work/seed.sql for review.	After changing built-in lesson content; review before creating/applying a migration.

Environment variables

Copy `.env.example` to `.env.local` only when setting up Supabase. Keep `.env.local` out of Git. The public browser can use only `NEXT_PUBLIC_SUPABASE_URL` and `NEXT_PUBLIC_SUPABASE_ANON_KEY`. The remaining keys are for Supabase server-side functions and must never be prefixed with NEXT_PUBLIC.

Variable group	Use
NEXT_PUBLIC_SUPABASE_URL / NEXT_PUBLIC_SUPABASE_ANON_KEY	Enables the browser Supabase client. Without both valid values, the app stays in demo mode.
RESEND_API_KEY / EMAIL_FROM / APP_URL / CRON_SECRET	Server-only reminder-email integration. Requires a provider and scheduling setup.
OPENAI_API_KEY	Server-only optional tutor integration. Requires a provider, consent workflow and live verification.

Supabase and deployment checklist

Step	Why it matters
1. Create your own Supabase project	The supplied source cannot provide a live database account for you.
2. Link the Supabase CLI and apply migrations 001–005 in order	Creates the database tables, policies, data and RPC functions.
3. Set public URL and anon key in .env.local	Lets the browser create a safe public client.
4. Configure Auth URL/redirect settings and email	Makes signup, confirmation and password recovery work on your deployed domain.
5. Deploy Edge Functions and add server secrets in Supabase	Needed only for account deletion, reminders and optional tutor paths.
6. Create synthetic adult test accounts	Verify login, progress, roles and data boundaries before real users.
7. Push source to your GitHub repository	GitHub becomes your source-of-truth for deploying from your account.
8. Import repository into Vercel	Use npm ci, npm run build and output directory out; add only public variables to Vercel.
9. Retest deployed site	Static export bakes public env values into the build, so rebuild after changing them.

Important: the existence of source files is evidence that integration code was implemented; it does not prove that a real email provider, AI service, or Supabase instance has been configured and tested.

8. Testing and validation

Testing answers two different questions: “does the code behave correctly in the tested case?” and “what still needs real-world verification?” It never proves every possible behavior.

Reported result	What it supports	What it does not prove
TypeScript type checking passed	The inspected TypeScript code matched its declared types at check time.	That every user journey or external API is correct.
ESLint passed	No configured lint rule failures were reported.	Full code review, security review or accessibility certification.
30 tests across 2 unit/database test files passed	Reward logic, restricted code/maze behavior, SQL migrations/RLS/RPC permissions and content validation were tested in the included suite.	A live hosted Supabase project behaves identically.
Production build succeeded	Next.js successfully produced static output.	Hosting, custom domain, email delivery or external secrets work.
Development server previously started at http://localhost:3000	The app was runnable locally in a browser.	It is public on the internet or configured for a particular deployment account.
npm audit --omit=dev reported zero vulnerabilities	The production dependency subset had no vulnerabilities reported by that command at the time of observation.	There are no risks in dev dependencies or future audits.
Full npm audit reported 5 high-severity vulnerabilities	There were unresolved audit findings in the complete dependency tree at the reported time.	That the app is necessarily exploitable; each advisory needs review, upgrade/testing and risk assessment.

The included automated browser journeys cover maze completion and duplicate XP, onboarding/settings persistence, filters, teacher demo class/assignment, mobile layout, main screen loading, dashboard accessibility scan, new games, circuit reset, demo ranking, content authoring/publishing, and missing account configuration handling.

Good next validation work

- Run npm audit and inspect each advisory before the expo; update only after tests pass.
- Create a separate Supabase test project and apply all migrations.
- Use adult synthetic accounts to test RLS, invitations, authentication redirects and deletion.
- Test email and optional tutor with dedicated keys and provider quotas.
- Use a real mobile phone and a screen reader for additional accessibility checks.

9. Expo preparation

Project abstract

STEMQuest is an interactive web platform designed to make STEM learning more engaging through short lessons, quizzes, educational games, simulations and progress feedback. It uses a React/Next.js interface for the learner experience and has a Supabase-based design for secure accounts, persistent progress and role-based access. The current build includes a browser-local demo so the main learning journey can be demonstrated without external credentials, while database migrations and server functions provide a path to production integration.

Problem statement

Many learners experience STEM topics as passive or abstract. Teachers also need a safe way to observe learning progress. STEMQuest addresses this by combining small interactive tasks, immediate explanations, motivational progress signals and an optional secure backend design.

Objectives

- Make STEM concepts approachable through interaction rather than only reading.
- Provide immediate feedback and explanations after learner choices.
- Track progress with XP, completed activities, streak information and achievements.
- Support different learning levels and subject worlds.
- Design an optional backend that protects answer keys and separates student, teacher, guardian and administrator access.
- Provide a credible demo mode for an expo without pretending it is a cloud database.

90-second presentation script

“Good morning. My project is STEMQuest, an interactive gamified STEM learning platform. The problem I focused on is that many STEM learning experiences are passive, while students benefit from immediate practice and feedback. STEMQuest presents short lessons across six worlds, then lets learners answer quizzes, solve a robot maze, play fraction games, explore gravity and circuits, and try a deliberately restricted Python loop exercise. As learners complete new activities, the app calculates XP, coins, streak days and achievements. For a reliable expo demo, the app works locally in the browser without API keys. For real deployment, I designed a Supabase backend with PostgreSQL migrations, row-level security and database functions. The important security point is that correct answers stay in a private database schema and rewards are validated on the server, not trusted from the browser. I tested the learning logic, database permissions and browser journeys. The main future work is live verification of Supabase Auth, email reminders and the optional AI tutor with real provider credentials. Thank you.”

10. Viva questions and answers

1. What is STEMQuest?

A gamified STEM learning web application with lessons, interactive activities, progress feedback and an optional secure backend.

2. Why did you choose Next.js?

It organizes the React app, supports a clear app structure and builds a static site that can be deployed easily.

3. Why use React components?

Each UI part can be reused and tested separately. For example, the game, dashboard and staff panels are separate components.

4. Why TypeScript instead of plain JavaScript?

It describes expected data shapes. This catches mistakes such as passing an invalid lesson or activity structure before runtime.

5. Is demo mode a real multi-user backend?

No. Demo mode is local to one browser. It is clearly separated from the Supabase-connected path.

6. Where is demo progress stored?

In browser localStorage, managed by the main StemQuest component and helper modules.

7. How do you prevent duplicate XP?

The demo engine checks completed activity keys. The database also has a unique student/lesson/activity constraint and awards XP transactionally.

8. Why keep answer keys private?

If answers were sent to the browser, a learner could inspect or alter them. The database private schema and RPC validation reduce that risk.

9. What is RLS?

Row Level Security is a PostgreSQL rule system. It lets the database restrict rows based on the signed-in user.

10. What is an RPC here?

A protected PostgreSQL function, such as submit_activity, that accepts an answer, checks it and returns correct/XP results.

11. How is the Python feature kept safe?

It does not execute arbitrary Python. It parses only a narrow range-loop and print(i) pattern and limits the number of iterations.

12. What does the robot maze teach?

Algorithms, sequence, debugging and testing. The learner builds a route from allowed commands.

13. What are the two science simulations?

GravityLab models fall time on Earth/Moon; CircuitLab models an ideal DC circuit with voltage, resistance, current and power.

14. How do tests help this project?

Vitest checks pure logic and embedded database rules. Playwright checks the visible user flows in a browser.

15. What are SQL migrations?

Numbered files that build or change the database in a repeatable, reviewable sequence.

16. Why use Supabase Edge Functions?

They keep server-only tasks, such as providers or privileged operations, away from browser code and secret exposure.

17. Is the AI tutor fully live?

No. The optional server-side integration exists, but it needs a real API key, consent process and live testing.

18. How could the project scale?

Use hosted PostgreSQL, efficient indexes already present for progress/XPs, pagination/aggregation for larger data, monitoring and a production deployment pipeline.

19. What is the biggest current limitation?

The main demo is local; live external services and real institutional workflows still require configuration and verification.

20. What would you build next?

More curriculum and authoring tools, teacher analytics, robust live QA, accessibility review, provider integration tests and production monitoring.

21. Why are there both local and server answer checks?

Local checks give instant demo feedback; server checks are the trusted production authority when Supabase is connected.

22. What does static export mean?

Next.js builds HTML/JS/CSS files that can be hosted as files. The backend services are separate Supabase services.

11. One-page revision sheet

Use this the evening before your expo.

Say this	Remember this
Project purpose	Interactive STEM learning with immediate practice, feedback and progress signals.
Main frontend	Next.js + React + TypeScript. page.tsx renders StemQuest; stemquest.tsx controls screens and state.
Content	content.ts defines worlds/lessons; extra-content.ts adds game lessons; types.ts defines data shapes.
Learning logic	engine.ts validates demo answers, runs maze/restricted Python, calculates XP/coins/streaks.
Demo truth	Works without credentials; localStorage only; not secure multi-user persistence.
Connected truth	supabase.ts creates optional client; database RPCs/RLS validate records and keep answers private.
Database	001–005 SQL migrations create schema, RLS, seeds, services, expanded content and authoring/class functions.
Server functions	Account, reminders and tutor Edge Functions are implemented but need secrets/provider setup.
Testing	30 unit/database tests plus browser journeys; lint/typecheck/build passed. Live services still need verification.
Best security line	Never trust browser XP or answer keys in production. Validate rewards on the database server.
Best limitation line	This is a strong working first release; live Supabase/Auth/email/AI need external setup and real verification.

Demo order for judges: 1) dashboard, 2) Robot rescue, 3) fuel or memory game, 4) circuit lab, 5) progress/leaderboard, 6) show code architecture in VS Code, 7) explain the optional secure Supabase path.

You do not need to memorize every file. Memorize the flow: **page** → **StemQuest controller** → **components** → **engine/content** → **local demo or Supabase database.**